Zeichen und Zeichenketten

Zeichen: Datentyp char

Merke: char ist in C ein ganzzahliger Datentyp

```
char ch = 'x';
```

Zeichenvariable

Zeichenkonstante

Zeichen braucht man zur Verarbeitung von Texten.

Zeichencodes

ASCII (American Standard Code for Information Interchange)

- 1 Zeichen = 1 Byte (128 bzw. 256 Zeichen darstellbar)
- z.B. in Pascal oder C verwendet

Unicode (www.unicode.org)

- 1 Zeichen = 2 Bytes (65536 Zeichen darstellbar)
- z.B. in Java und C# verwendet
- ASCII ist Teilmenge von Unicode

ASCII-Code

0	NUL	16	DLE	32	space	48	0	64	@	80	P	96	´	112	p
1	SOH	17	DC1	33	!	49	1	65	A	81	Q	97	a	113	q
2	STX	18	DC2	34	"	50	2	66	B	82	R	98	b	114	r
3	ETX	19	DC3	35	#	51	3	67	C	83	S	99	c	115	s
4	EOT	20	DC4	36	$	52	4	68	D	84	T	100	d	116	t
5	ENQ	21	NAK	37	%	53	5	69	E	85	U	101	e	117	u
6	ACK	22	SYN	38	&	54	6	70	F	86	V	102	f	118	v
7	BEL	23	ETB	39	'	55	7	71	G	87	W	103	g	119	w
8	BS	24	CAN	40	(	56	8	72	H	88	X	104	h	120	x
9	HT	25	EM	41	)	57	9	73	I	89	Y	105	i	121	y
10	LF	26	SUB	42	*	58	:	74	J	90	Z	106	j	122	z
11	VT	27	ESC	43	+	59	;	75	K	91	[	107	k	123	{
12	FF	28	FS	44	,	60	<	76	L	92	\	108	l	124	|
13	CR	29	GS	45	-	61	=	77	M	93	]	109	m	125	}
14	SO	30	RS	46	.	62	>	78	N	94	^	110	n	126	~
15	SI	31	US	47	/	63	?	79	O	95	_	111	o	127	DEL

Beispiele einiger Steuerzeichen:

BS backspace	löscht Zeichen vor Cursor	CR carriage return	Zeilenvorschub
HT horizontal	tab Tabulatorsprung	LF line feed	folgt auf CR

Unicode

0000 – 007F	ASCII-Zeichen
0080 – 024F	Umlaute, Akzente, Sonderzeichen
0370 – 03FF	griechische Zeichen
0400 – 04FF	kyrillische Zeichen
0530 – 058F	armenische Zeichen
0590 – 05FF	hebräische Zeichen
0600 – 06FF	arabische Zeichen
...	...

Details siehe
http://www.unicode.org

Deutsche Umlaute

00E4	ä	00C4	Ä	00DF ß
00F6	ö	00D6	Ö	
00FC	ü	00DC	Ü	

Programmiersprache C: Zeichensatz

- C-Standards unterscheiden zwischen Quell- und Ausführungszeichensatz
- beide Zeichensätze bestehen aus einem Basiszeichensatz (darstellbar mit 1 Byte) und einem länderspezifischen Zeichensatz
- Zeichen des Basis-Quell- und Basis-Ausführungszeichensatzes:
  - die zehn Dezimalziffern: 1 2 3 4 5 6 7 8 9 0
  - die Buchstaben des englischen Alphabets:

    A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

    a b c d e f g h i j k l m n o p q r s t u v w x y z
  - folgende Grafiksymbole: ! " % & / ( ) [ ] { } \ ? = ’ # + * ~ – _ . : ; , | < > ^
- zusätzlich im Basis-Ausführungszeichensatz: Steuerzeichen für Return (\r), neue Zeile (\n), Tabulator (\t), Backspace (\b), Formfeed (\f), Backslash (\\)
- die länderspezifischen Zeichensätze sind in C implementierungsabhängig
- Praxis: C-Compiler verwenden zur Zeichendarstellung ASCII-Format
- Datentypen zur Unicode-Darstellung:
  - wchar_t, seit Standard-C11 auch char_16_t und char_32_t

Zeichen sind in C Zahlen

Datentyp *char* steht in C für den kleinsten ganzzahligen Datentyp

- Zeichen werden intern als ganze Zahlen beschrieben

Zuweisungen

```
char ch1, ch2 = 'a';
ch1 = ch2;    // ok, gleicher Typ
int i = ch2;  // ok, kein Informationsverlust
ch1 = i;      // ok, aber Informationsverlust möglich
```

Vergleiche (==, !=, <, <=, >, >=)

Zeichen sind nach ASCII-Code-Werten geordnet;
Buchstaben und Ziffern liegen aufeinanderfolgend

```
if ('a' <= ch && ch <= 'z' || 'A' <= ch && ch <= 'Z') ...
```

Arithmetische Operationen (+, -, *, /, %)

```
char ch = 'A' + 1; printf("%c", ch); // Ausgabe: B
```

Standard-Funktionen auf Zeichen

<ctype.h> definiert diverse Funktionen zur Bearbeitung von Zeichen

int isalnum(int c)	testet auf alphanumerisches Zeichen (a-z, A-Z, 0-9)
int isalpha(int c)	testet auf Buchstabe (a-z, A-Z)
int iscntrl(int c)	testet auf Steuerzeichen (\f, \n, \t, ...)
int isdigit(int c)	testet auf Dezimalziffer (0-9)
int isgraph(int c)	testet auf druckbare Zeichen ohne Leerzeichen
int islower(int c)	testet auf Kleinbuchstaben (a-z)
int isupper(int c)	testet auf Grossbuchstaben (A-Z)
int isxdigit(int c)	testet auf hexadezimale Ziffern (0-9, a-f, A-F)
int tolower(int c)	wandelt Gross- in Kleinbuchstaben um
int toupper(int c)	wandelt Klein- in Grossbuchstaben um

Zeichenketten in C

Zeichenketten in C

- in C gibt es keinen eigenen Typ für Zeichenketten, anstatt dessen werden diese als Reihungen von Zeichen definiert
- um das Ende einer Zeichenkette zu markieren, endet eine Zeichenkette immer mit dem Nullzeichen '\0'

```
char buffer[13] = {'Z', 'e', 'i', 'c', 'h', 'e', 'n', 'k', 'e', 't', 't', 'e', '\0'};
char buffer[13] = "Zeichenkette";
char buffer[13] = {"Zeichenkette"};
```

- die Endung '\0' (Nullzeichen) wird zum vollständigen Durchlaufen von Zeichenketten verwendet

Merke: Formatierungszeichen %s steht für Zeichenketten

```
int length; char buffer[] = "Hello";
for(length = 0; buffer[length] != '\0'; length++) ;
printf("Laenge der Zeichenkette %s ist %d \n", buffer, length);
```

- Achtung: eine manuell angelegte Reihung zur Darstellung von Zeichenketten, muss immer als letztes Element die '\0' enthalten

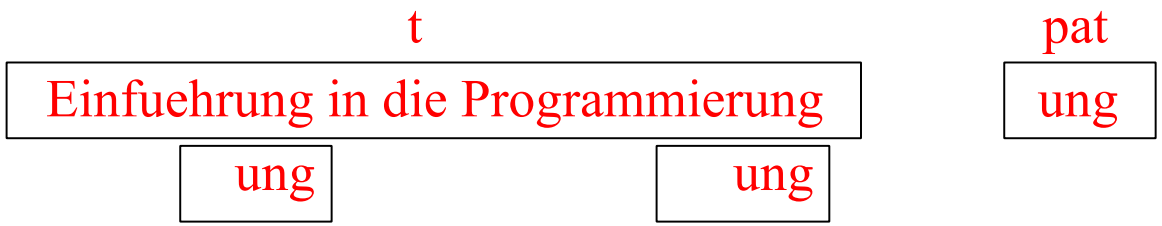
Beispiel: Zeichenketten

```
/* Datei: char_array.c */
/* Ueberpruefen, ob eine eingelesene Zeichenkette ein 'a' enthält */

#include <stdio.h>
#include <string.h>
#define MAX 40
int main (void){
      int index;
      char eingabe [MAX+1];
      printf ("Bitte String eingeben (max. %d Zeichen): ", MAX);
      gets (eingabe);
      for (index = 0; eingabe[index] != '\0'; index++){
            if (eingabe[index] == 'a')
                  break;
      }
      if (eingabe[index] == '\0')
            printf ("\nIhr String enthaelt kein 'a'\n");
      else
            printf ("\nDas a befand sich an der %d. Stelle\n", index + 1);
      return 0;
}
```

Beispiel: Textsuche

geg.: Text t, Muster pat
ges.: erstes Vorkommen von pat in t




```
int search (char t[], char pat[]) {
    int last = strlen(t) – strlen(pat);
    for (int i = 0; i <= last; i++) {
        if (t[i] == pat[0]) {
            int j = 1;
            while (j < strlen(pat) && pat[j] == t[i+j]) j++;
            // j == pat.length || pat[j] != t[i+j]
            if (j == strlen(pat)) return i;
        }
    }
    return -1;
}
```

Ergebnis:

pos ≥ 0: pat in t an Stelle pos

pos < 0: pat kommt nicht in t vor

Verwendung von strlen() verlangt #include <string.h>

Beispiel: Zeichenketten von Hand kopieren

```
char alpha [30] = "zu kopierender String";
char beta [30] = "";

    int i = 0;
    while(alpha[i] != '\0'){          // alle Zeichen bis auf '\0' werden kopiert
        beta[i] = alpha[i]; i++;
    }
    beta[i] = '\0';
```

etwas kürzer

```
    int i = 0;
    while((beta[i] = alpha[i]) != '\0') // alle Zeichen bis zu '\0' werden kopiert
            i++;
```

noch knapper

```
    while(beta[i] = alpha[i])    // alle Zeichen bis zu '\0' werden kopiert
            i++;
```

Beispiel: Zeichenketten (mit Zeigern) von Hand kopieren

```
char *ptralpha = alpha;          // alpha und beta sind Reihungsvariablen
char *ptrbeta = beta;            // und können nicht geaendert werden

    while(*ptralpha != '\0'){        // alle Zeichen bis auf '\0' werden kopiert
        *ptrbeta = *ptralpha;
        ptralpha++;
        ptrbeta++;
    }
    *ptrbeta = '\0';
```

etwas kürzer

```
    while(*ptralpha != '\0'){        // alle Zeichen bis auf '\0' werden kopiert
        *ptrbeta++ = *ptralpha++;
    }
    *ptrbeta = '\0';
```

noch knapper

```
    while(*ptrbeta++ = *ptralpha++) // alle Zeichen bis '\0' werden kopiert
            ;
```